Capítulo 4: Álgebra de tablas y su implementación

Introducción

Tabla de contenidos

- Bloque 1: Cómo ejecutar una consulta SQL 3
- Bloque 2: Álgebra de tablas 8
  - Cómo especificar un operador sobre álgebra de tablas 10
- Bloque 3: Implementación de operadores 12
  - Implementación física 14

Bloque 1: Cómo ejecutar una consulta SQL

- **Contexto**:
  - Una consulta pasó los análisis sintáctico y semántico con éxito.
  - La consulta necesita ejecutarse para obtener la información deseada.
- Para **analizar** cómo ejecutar una consulta **ayudan las recetas** en lenguaje natural **sobre cómo se ejecutan las consultas.**
- Al final de la unidad veremos que las ideas gruesas para construir un SGBD se pueden trasladar a otros ámbitos de la computación más allá de las bases de datos.
  - Por ejemplo sistemas de recuperación de la información.

Bloque 1: Cómo ejecutar una consulta SQL

Contexto

- Las consultas SQL suelen ser muy complejas. Por ejemplo:
  - SELECT …
  - FROM …
  - WHERE …
  - GROUP BY ….
  - HAVING …
  - ORDER BY…
- Esto sin contar que además puede haber subconsultas anidadas con la misma estructura (por ejemplo dentro del WHERE, HAVING o FROM).
- Sin embargo al describir informalmente el significado de una consulta solemos hacerlo como una secuencia de pasos.

Bloque 1: Cómo ejecutar una consulta SQL
Ejecutar consulta mediante secuencia de pasos

- **Por ejemplo**:

  SELECT nombre FROM Profesor, Enseña

  WHERE salario > 10000 and Profesor.DNI = Enseña.DNI

  - Tiene el significado:
    1. Obtener los registros (p1…pn, e1,…,em) donde (p1,…,pn) tupla de profesor y (e1,…,em) tupla de enseña.
    2. Dejar solo las tuplas que cumplen la propiedad del WHERE.
    3. De las tuplas resultantes extraer solo los nombres obteniendo una table de los nombres de profesores esperados.

- **Problema**: ¿Cómo representar formalmente esos pasos?

Bloque 1: Cómo ejecutar una consulta SQL
Cada paso transforma información

- Observamos una característica importante. Cada paso recibe información en forma de tablas. Cada paso produce nueva información en forma de una tabla.
- **Solución**: Esto nos sugiere que cada paso puede verse como un operador.
- **Por ejemplo:**
  - El paso 1 es hacer una especie de producto cartesiano de dos tablas.
  - El paso 2 es hacer una selección de tuplas a partir de una table y un operador.
  - El paso 3 es hacer lo que en matemática se llama proyección, proyectar atributos desde una table.

Bloque 1: Cómo ejecutar una consulta SQL
Cada paso transforma información

- En SQL las consultas trabajan sobre tablas y producen nuevas tablas.
- Luego los operadores que necesitamos definir son operadores que transforman una o más tablas en una tabla.
- De aquí surge naturalmente la idea de un **Álgebra de Tablas**.

Bloque 2: Álgebra de tablas
Contexto

- Las **tablas de SQL** mantienen un orden, pueden contener tuplas repetidas y se representan naturalmente como listas de registros.
- Una consulta SQL se puede desmenuzar como un conjunto de operaciones sobre tablas.
  - Esas operaciones básicas se ejecutan en un cierto orden.
- SQL no da la semántica de esas operaciones.

Bloque 2: Álgebra de tablas
Diferencia entre las tablas de SQL y las del modelo relacional

- El modelo relacional trabaja con tablas que son relaciones; o sea, conjuntos de tuplas.
- El **álgebra relacional** al trabajar con el modelo relacional trabaja con operandos que son relaciones.
  - Esto quiere decir que a diferencia de las tablas de SQL en una relación no hay un orden entre los elementos, ni puede haber tuplas repetidas.
  - Como consecuencia de esto, el algebra relacional no cuenta con algunas operaciones importantes.
    - **Por ejemplo**: ordenar una tabla según un conjunto de atributos, remover duplicados de una tabla, unir dos tablas generando tuplas duplicadas.

- En cambio para acercarnos más a la semántica de SQL, nosotros no trabajaremos sobre relaciones.
  - Vamos a considerar una tabla como una **lista de tuplas**.
  - Por eso, vamos a definir **operadores del álgebra de tablas sobre listas de tuplas.**
  - Esto permite **usar programación funcional** para definir operaciones sobre tablas.
  - Además se pueden **probar propiedades de las operaciones** del álgebra de tablas **por inducción sobre listas**

Bloque 2: Álgebra de tablas
Cómo especificar un operador sobre tablas

- **¿Qué operadores de álgebra de tabla hacen falta?**
- Aquellos correspondientes a los pasos de la semántica de una consulta SQL.
- Un operador del álgebra de tablas también se llama **operador lógico** porque se está en el nivel lógico.
- **Problema**: ¿Cómo definir precisamente el comportamiento de un operador del álgebra de tablas?
- **Solución**: Ustedes vieron en materias anteriores una forma de definir operadores sobre listas:
  - Programación funcional sobre listas, o
  - **definiciones recursivas** sobre listas.

Bloque 2: Álgebra de tablas
Cómo especificar un operador sobre tablas

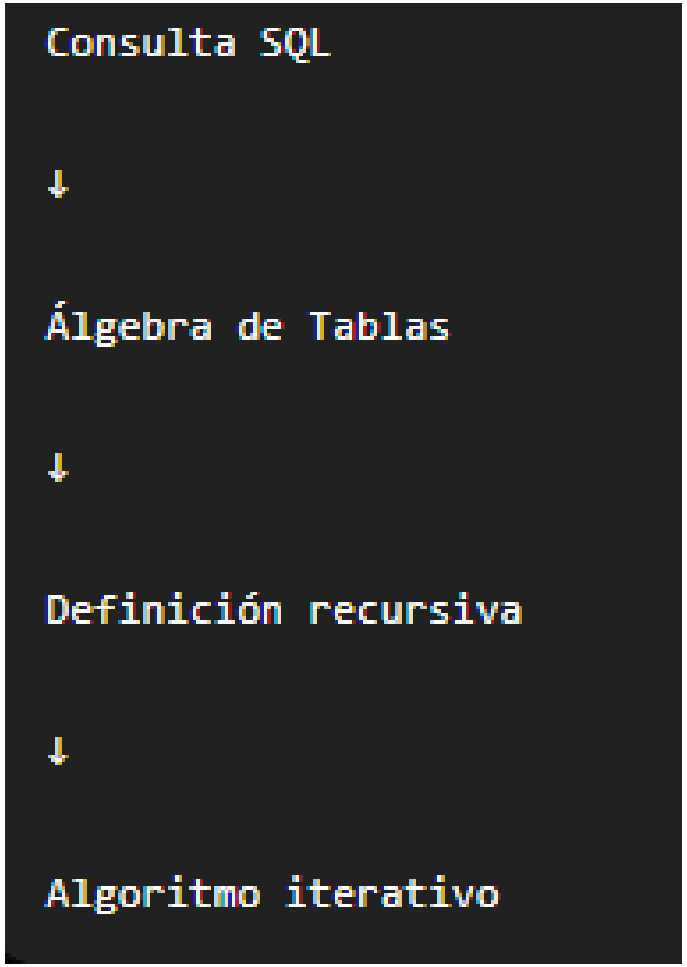
- **Por ejemplo**: Definimos el operador selección de tuplas que cumplen un predicado p sobre una tabla así:
  - selección p, [] = []
  - Selección p, x::l = if p(x) then x :: selección p, l else selección p, l.
- Esta definición describe exactamente el comportamiento esperado del operador, sin hablar todavía de archivos, índices o bloques.

Bloque 3: Implementación de operadores Contexto

- SQL no dice como implementar las operaciones del álgebra de tablas.
- Contamos con un gestor de almacenamiento con primitivas para leer y escribir archivos y acceder a índices.
- Los lenguajes de programación contienen primitivas para trabajar en memoria.
  - p.ej: manejo de variables, definición de tipos, primitivas para procesar estructuras de datos.
- No tenemos una manera de evaluar el costo de una implementación de una operación sobre el álgebra de tablas.

Bloque 3: Implementación de operadores
Del operador al algoritmo

- Las definiciones recursivas constituyen una especificación.
- A partir de ellas podemos obtener implementaciones iterativas.
  - Estas acceden a los datos
- **Problema:** ¿Cómo accede un operador del álgebra de tablas a los datos?



Bloque 3: Implementación de operadores Implementación física

- **Solución**: Los algoritmos ya no trabajan directamente sobre tablas abstractas.
  - Utilizan las primitivas ofrecidas por el gestor de almacenamiento.
  - Utilizan primitivas sobre estructuras en memoria.
  - La implementación de un operador se llama **operador físico** porque se está en el nivel físico.
- Por ejemplo, para

  selección A = c, r
  - Podemos recorrer la tabla secuencialmente para obtener el resultado.
  - Tenemos así:

```
SelecciónSecuencial(r, R, A, c)
  if not  exist(f): then create(f, R)
  Buffer_salida = vacío
  B = leer_primer_bloque(r)
  While B ≠ null do
    For t in B do
        if t[A] = c then
            if no_hay_lugar(buffer_salida)
              write(buffer_salida,f)
              buffer_salida = vacío
            fi
            insertar(t, buffer_salida) fi
      od
      B = Leer_siguiente_bloque(r)
  od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

Bloque 3: Implementación de operadores Implementación física

- **El ejemplo anterior permite distinguir dos niveles:**
  - Primitivas del gestor de almacenamiento
    - Leer_primer_bloque(r)
    - Leer_siguiente_bloque(r)
    - Insertar(t, B)
    - No_hay_lugar(B)
    - Exist(f)
    - Create(f,R)
    - Write(B, f)
  - Primitivas sobre estructuras en memoria
    - Evaluar t[A] = c

- **El costo del operador se deduce directamente**
  - Si r ocupa B(r) bloques, el recorrido secuencial lee cada bloque una vez:
  - Costo = B(r) transferencias de bloques
  - Asumimos que el resultado solo se mantiene en memoria.

Bloque 3: Implementación de operadores
¿Existe una única implementación para un operador?

- Hasta ahora vimos una implementación para *selección(A=c, r)* mediante un recorrido secuencial de la tabla.
- Surge la pregunta: **¿es la única manera de implementar una selección?**
- Supongamos que existe un índice sobre el atributo A.
  - Entonces no es necesario recorrer toda la tabla.
- Podemos:
  - Buscar c en el índice,
  - obtener las referencias a las tuplas,
  - leer únicamente esas tuplas.
- Tenemos entonces **otro operador físico** para la misma operación lógica.

Bloque 3: Implementación de operadores
¿Existe una única implementación para un operador?

- **Conclusión**: Una operación lógica puede tener varias implementaciones.
  - Cada implementación produce exactamente el mismo resultado.
  - Lo que cambia es el costo.
- **Por ejemplo**: como veremos más adelante, usar el índice va a hacer bajar el costo de la consulta anterior.